

Tutorial 7

CSE 112 Computer Organization

Q1.

Macros are placeholders for a given piece of code. If an assembly program uses macros, the macros will be replaced by their corresponding code by the assembler. Functions on the other hand are “called”. This means that the function arguments and return address needs to be stored somewhere (usually on the stack) on each call. There is no such need in case of macros and they are not “called”. Macros are merely replaced by their definitions in the code.

Since macros are simply replaced by their definition, they get effectively inlined. This prevents the storing and restoring of context information upon a call. As a result, macros are usually more performant as compared to functions.

Q2.

Caller-Callee conventions are a set of rules followed by the compiler while implementing functions. These conventions outline the compiler behavior regarding where/how arguments are placed, where/how return address is stored, which all registers should be preserved by the caller and callee, etc.

If all of the source code is available to the compiler statically, then the compiler can come up with its own custom conventions. In this case, the caller-callee conventions are not strictly necessary.

However, often a lot of pre-compiled libraries are linked with the source code (both statically and dynamically). Therefore there is a need to strictly define the interface these libraries provide. This interface is encapsulated in the caller-callee conventions.

The most important rules that a caller-callee convention outlines are:

1. Where are the input arguments stored by the caller before callee is called?
2. What order are the arguments stored in?
3. Where is the return address stored by the caller before calling the callee?
4. How does the callee return data to the caller?
5. Which all registers must be preserved by the caller across a function call? 6. What all

registers must be preserved by the callee across a function call? There needs to be a consensus regarding who preserves what registers as each function assumes access to all registers. If there is no consensus, then it can happen that the callee thrashes a register that the caller was using without the knowledge of the caller.

Q3:

Approximately 50-50 split. Since each function is equally likely to each register, giving preference to either callee or the caller is not correct.

Suppose that all registers are caller saved. In this case, we will end up saving a lot of registers conservatively on each call even if the callee is not using those registers. On the other hand if only half of the registers are caller saved, the potential to “over-save” is less.

Similarly “under-saving” forces the callee to act conservatively as it limits the number of available registers which the callee can use, leaving the function called a very few options in terms of number of registers to exploit for its operation.

Therefore when no other information is available, a 50-50 split between caller and callee saved registers is most performant.

Q4:

push RX:

```
sub R15, $1
st RX, [R15]
```

pop RX:

```
ld RX, [R15]
add R15, $1
```

call RX:

```
mov R2, PC
add R2, $1
push R2
br [RX]
```

ret:

```
pop R1
br [R1]
```

Q5:

baz:

```
add r2 r3 r4
mov r15 r1
```

bar:

```
mov r3 $1
mov r4 $2
push r1
brl baz
pop r1
mov r15 r1
```

foo:

```
push r1
brl bar
pop r1
mov r15 r1
```

Q6

1.

r1 => LR
 ro => SP
 r15 => PC

16. mov r2 100 //r2 =100
 17. mov r3 \$10 //r1=10
 18. mov r4 \$15 //r4=15
 19. push r1
 20. Push r2

100
<main_return_address>

21. brl foo //LR = 22

1. add r5 r3 r4 //r5 = 10+15 = 25
 2. mov r3 \$100 //r3 =100
 3. r4 = 80
 4. push r1
 5. push r5

25
22
100
<main_return_address>

6. brl bar // LR = 7 => r1 = 7

11. push r7

r7
25
22
100

<main_return_address>

12. sub r7 r3 r4 //r7 = 100-80 = 20

13. mov r2 r7 //r2 = 20

14. pop r7

25
22
100
<main_return_address>

15. mov r15 r1 //r15 = 7

7. pop r5 //r5 = 25

8. pop r1 //r1=22

100
<main_return_address>

9. mov r2 r5 //r2=25

10. mov r15 r1 //r15 =22 => PC=22

22. pop r2 //r2=100

<main_return_address>

23. pop r1 //r1=LR0

--

24. mov r15 r1 //PC,r15=LR0

Before 21,

100
<main_return_address>

Before 6,

25
22
100
<main_return_address>

After 12,

r7
25
22
100
<main_return_address>

After 14,

25
22
100
<main_return_address>

After 8,

100
<main_return_address>

After 23,

--

2.

```
int foo(int a, int b)
{
    int t = a+b;
    bar(100, 80)
    return t;
}
int bar(int a, int b)
{
    int t = a-b;
    return t;
}
void main()
{
    int t = 100;
    foo(10, 15);
    return t;
}
```

3.

Instruction 21: caller:main, callee: foo

Instruction 6: caller: foo, callee: bar